AI-Powered Malware Analysis Platform — Architecture Design

1. System Overview

A SOC-grade platform that accepts suspicious files/URLs, detonates them in isolated sandboxes, extracts static and behavioral indicators, runs AI classification, cross-references threat intelligence, and produces analyst- and executive-facing reports.

```
flowchart LR
    U[Analyst / API Client] --> GW[API Gateway - FastAPI]
    GW --> AUTH[Auth Service - RBAC/JWT]
    GW --> SUB[Submission Service]
    SUB --> QUEUE[(Redis/Celery Queue)]
    QUEUE --> STATIC[Static Analysis Worker]
    QUEUE --> DYNAMIC[Sandbox Orchestrator]
    DYNAMIC --> VM1[Windows Sandbox VM]
    DYNAMIC --> VM2[Linux Container Sandbox]
    STATIC --> FEATSTORE[(Feature Store)]
    DYNAMIC --> FEATSTORE
    FEATSTORE --> AI[AI Inference Service]
    AI --> TI[Threat Intel Correlator]
    TI --> EXTFEEDS[(External TI Feeds - VT/OTX/AbuseIPDB)]
    AI --> REPORT[Report Generator]
    REPORT --> DB[(PostgreSQL)]
    DB --> API2[Query/Report API]
    API2 --> FE[React + TS Dashboard]
    DB --> SIEM[SIEM/REST Export]
```

Design principles: every untrusted sample only ever touches network-isolated, ephemeral, snapshot-restored sandboxes; the control plane (API, DB, queue) never executes untrusted code; AI outputs are always paired with the evidence (features/IOCs) that produced them for explainability.

2. Database Schema (PostgreSQL, abridged)

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,           -- argon2id
    role TEXT NOT NULL CHECK (role IN ('admin','analyst','viewer','api_service')),
    mfa_secret TEXT,
```

```
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE samples (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sha256 TEXT UNIQUE NOT NULL,
    sha1 TEXT, md5 TEXT,
    file_type TEXT,                         -- PE, ELF, script, doc, archive, url
    size_bytes BIGINT,
    storage_path TEXT NOT NULL,             -- encrypted object storage key
    submitted_by UUID REFERENCES users(id),
    submitted_at TIMESTAMPTZ DEFAULT now(),
    status TEXT DEFAULT 'queued'            -- queued/analyzing/completed/failed
);

CREATE TABLE analysis_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sample_id UUID REFERENCES samples(id),
    sandbox_profile TEXT,                   -- win10-x64, ubuntu22-x64, etc.
    started_at TIMESTAMPTZ, finished_at TIMESTAMPTZ,
    exit_status TEXT
);

CREATE TABLE static_features (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sample_id UUID REFERENCES samples(id),
    entropy FLOAT,
    imports JSONB, sections JSONB, strings JSONB,
    yara_matches TEXT[]
);

CREATE TABLE behavioral_events (
    id BIGSERIAL PRIMARY KEY,
    job_id UUID REFERENCES analysis_jobs(id),
    ts TIMESTAMPTZ, event_type TEXT,        -- file/registry/process/network/api
    detail JSONB
);

CREATE TABLE iocs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id UUID REFERENCES analysis_jobs(id),
    ioc_type TEXT,                           -- ip, domain, hash, mutex, registry_key
    value TEXT,
```

```
    source TEXT                             -- static | dynamic | ti_enrichment
);

CREATE TABLE ai_verdicts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id UUID REFERENCES analysis_jobs(id),
    threat_score FLOAT CHECK (threat_score BETWEEN 0 AND 100),
    malware_family TEXT,
    confidence FLOAT,
    mitre_techniques TEXT[],
    explanation JSONB,                      -- feature attributions (SHAP), top signals
    model_version TEXT
);

CREATE TABLE reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id UUID REFERENCES analysis_jobs(id),
    summary_technical TEXT,
    summary_executive TEXT,
    remediation JSONB,
    pdf_path TEXT,
    generated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit_log (
    id BIGSERIAL PRIMARY KEY,
    actor_id UUID, action TEXT, target TEXT,
    ts TIMESTAMPTZ DEFAULT now(), ip TEXT
);
```

Indexes: `samples(sha256)`, `behavioral_events(job_id, ts)`, `iocs(value)` for fast TI lookups, partitioning `behavioral_events` by month for retention.

3. REST API Design (FastAPI)

Method	Endpoint	Purpose
POST	`/api/v1/samples`	Upload file/URL, returns `sample_id`, enqueues job
GET	`/api/v1/samples/{id}`	Sample metadata + status
GET	`/api/v1/jobs/{id}/events?type=network`	Streamed/paged behavioral events
GET	`/api/v1/jobs/{id}/report`	Full JSON report
GET	`/api/v1/jobs/{id}/report.pdf`	Exported PDF

Method	Endpoint	Purpose
GET	`/api/v1/ioc/lookup?value=`	Cross-reference IOC against TI + historical corpus
POST	`/api/v1/yara/rules`	Manage custom YARA rules (admin/analyst)
GET	`/api/v1/dashboard/stats`	Aggregate threat metrics for dashboard
WS	`/ws/jobs/{id}`	Live analysis event stream to UI
POST	`/api/v1/auth/login`, `/refresh`, `/mfa/verify`	AuthN

All mutating endpoints require a per-session CSRF token; all endpoints enforce ownership/role checks server-side (404, not 403, for resources outside the caller's scope); file upload endpoint enforces size limits, MIME/magic-byte checks, and immediately quarantines to a non-executable, non-webroot bucket before any processing.

4. AI Analysis Pipeline

1. **Feature extraction** — static (PE/ELF headers, imports, entropy, string n-grams) + dynamic (API call sequences, process tree shape, network beacon patterns) merged into a unified feature vector per sample.
2. **Family classification** — gradient-boosted model (Scikit-learn) on static features for fast triage, cascaded into a transformer-based sequence model (PyTorch/Transformers) over the API-call/behavioral event sequence for deeper classification when the fast path is uncertain.
3. **Malicious-intent scoring** — ensemble of the above plus rule-based YARA/heuristic signals, combined into a calibrated 0–100 threat score (isotonic regression calibration, not a raw softmax, so the number reflects real confidence).
4. **Explainability** — SHAP values for the tree model, attention-weight highlighting for the sequence model, surfaced as "top contributing signals" in `ai_verdicts.explanation`; the UI must show *why*, not just the score, for every verdict.
5. **MITRE ATT&CK mapping** — behavioral events matched against a technique-signature table (e.g., registry Run-key write → T1547.001) maintained as versioned, human-reviewable mapping data, not model-inferred.
6. **Human-in-the-loop** — verdicts below a confidence threshold, or flagging novel behavior clusters, are routed to an analyst review queue rather than auto-published.

5. Sandbox Orchestration

- **Isolation:** each job gets a freshly cloned VM (Windows) or gVisor/Firecracker microVM (Linux) from a golden snapshot; network is routed through an inline monitoring proxy (INetSim/simulated internet) — never real internet access — to capture C2/DNS behavior safely while preventing actual harm.
- **Instrumentation:** kernel-level hooks (ETW on Windows, eBPF on Linux) capture process, file, registry, and network events without relying on in-guest agents that malware could detect/kill.

- **Anti-evasion basics:** randomized VM fingerprints, human-interaction simulation (mouse/keyboard jitter), delayed-execution handling (timeout tuned per sample class).
- **Teardown:** VM/container destroyed and snapshot reverted after each run — no state ever persists between samples.
- **Queueing:** Celery workers pull jobs from Redis; sandbox pool size auto-scales via Kubernetes HPA based on queue depth; each worker only ever talks to its own isolated sandbox network segment.

6. Frontend (React + TS + Tailwind)

- **Upload page** — drag/drop, URL submission, batch upload, submission-time hash pre-check against existing corpus to skip redundant analysis.
- **Live analysis monitor** — WebSocket-driven real-time event feed, process tree graph (D3.js), network connection map.
- **Threat intel dashboard** — global stats, trending families, IOC search, campaign clustering view.
- **Report viewer** — tabbed technical/executive views, MITRE ATT&CK matrix heatmap, PDF export.
- **Analyst workspace** — side-by-side static/dynamic evidence, verdict override with justification (feeds back into model retraining queue), case notes.
- Dark theme with severity color scale (e.g., gray→green→yellow→orange→red mapped to the 0–100 score) applied consistently across all charts.

7. Security Controls

- RBAC (admin/analyst/viewer/api_service) enforced at the API layer, never trusting frontend role display.
- Samples encrypted at rest (per-object keys via KMS), stored outside any web-servable path.
- Sandbox network fully air-gapped from production network; separate VPC/subnet with explicit deny-by-default egress except to the simulated-internet proxy.
- Secrets (TI API keys, DB credentials) in a secrets manager, never in code or images.
- All API responses use context-appropriate output encoding; JWT with short expiry + rotation; MFA required for admin/analyst roles.
- Full audit logging of sample access, verdict overrides, and rule changes.
- Rate limiting and per-tenant quotas to prevent sandbox pool exhaustion (DoS).

8. Deployment Strategy

- Kubernetes-based: API/frontend/AI-inference as stateless deployments with HPA; sandbox workers as a separately-scaled, network-isolated node pool (bare-metal or nested-virt nodes for Windows VMs, gVisor/Firecracker nodes for Linux).
- PostgreSQL managed with read replicas for dashboard queries; Redis for queue + caching.
- Object storage (S3-compatible) for samples/reports with lifecycle policies (auto-archive/delete per retention policy).

- CI/CD: model artifacts versioned and deployed independently from application code, with a shadow-mode evaluation step before a new model version affects live verdicts.
- Observability: centralized logging/metrics (Prometheus/Grafana), plus sandbox-specific health checks (snapshot integrity, queue depth, worker crash rate).

9. Scalability & Explainability Summary

Horizontal scaling on both the sandbox pool and AI inference layer decouples throughput from the control plane. Every AI verdict carries its evidence trail (feature attributions + matched IOCs/techniques) so analysts can validate rather than blindly trust automated scoring — this is treated as a hard requirement, not a nice-to-have, given the cost of a wrong verdict in a SOC workflow.